ENGINEERING

McMaster University

Lab 5

ELECENG 3EY4

Alaa Abdelsalam – abdela74 – 400396373

Tina Ismail – ismait1 – 400299939

Marisa Patel – patem156 – 400393635

February 29, 2024

Table of Contents

Objective 1: Review of the Lab Manual

Contribution

Objective 2: Alaa

Objective 3: Tina

Objective 4: Marisa

Objective 5: Alaa, Tina, Marisa

Objective 2: VESC Driver Setup

Question 1

The q-axis has an intrinsic relationship to torque production, as observed during no-load conditions all voltage is reflected on the q axis since when we apply current on the q-axis it results in a torque, referred to as the torque-producing-current in the permanent magnet synchronous motor in the McMaster AEV. The q-axis current is proportional to the torque generated by the motor.

The role of q-axis current in the operation of a permanent magnet synchronous motor (PMSM) is critical, primarily impacting torque generation. This current component corresponds to the portion of the motor current that aligns with the magnetic flux generated by the permanent magnets within the motor. As it interacts with the magnetic field of the rotor's permanent magnets, torque is produced in the motor. Adjusting the q-axis current directly influences torque generation: an increase strengthens the stator's magnetic field, resulting in higher torque output, while a decrease leads to a reduction in torque output. Therefore, precise control over the q-axis current allows for fine-tuning of the torque produced by the motor, facilitating tasks such as speed regulation, position control, and torque control across various applications, including electric vehicles like the McMaster AEV.

Question 2

Installing the VESC package with apt-get and cloning the repository from GitHub are two entirely different actions. Using apt-get simply installs all the necessary packages required to run the project successfully on your local machine. After which, the GitHub repository must be cloned in order to have a working environment from which to run on.

Question 3

In the code, ‘static_cast’ is used to convert raw data bytes from the VESC to meaningful data types that can be used to represent the q-axis current value. In this instance, we used ‘static_cast’ to convert from raw bytes from the VESC into a 32-bit integer (‘int32_t’). This is necessary for the data to be read in the appropriate data type and be parsed and visualized accurately. The utilization of ‘static_cast’ ensures the accurate interpretation and conversion of the raw byte data from the VESC into the appropriate data type. This enables the driver software to effectively decipher and utilize the q-axis current data for tasks related to control and monitoring.

Question 4

“double VescPacketValues::current_gaxis() const”: Defines a function named current_gaxis() in the VescPacketValues class. This function contributes to parsing the q-axis current data as a double.

“int32_t v = static_cast<int32_t>((static_cast<uint32_t>(*(payload_.first + 17)) << 24) +
(static_cast<uint32_t>(*(payload_.first + 18)) << 16) +
(static_cast<uint32_t>(*(payload_.first + 19)) << 8) +
static_cast<uint32_t>(*(payload_.first + 20)));”

This code excerpt demonstrates the process of parsing the q-axis current data. To retrieve the byte at position 17 in the payload, which contains the most significant byte (MSB) of the q-axis current data, we use the expression *(payload_.first + 17). The byte is then cast to an unsigned 32-bit integer and shifted left by 24 bits with the expression static_cast<uint32_t>(*(payload_.first + 17)) << 24. This shifts the byte to the most significant position in the 32-bit integer. The same is done with the bytes at positions 18, 19, and 20, which are retrieved and shifted left by 16, 8, and 0 bits respectively to place them in their appropriate positions within the 32-bit integer. We then combine the shifted bytes using bitwise OR (+) operations to form a 32-bit integer value representing the q-axis current. Finally, we store this integer value in the variable v. “return static_cast<double>(v) / 100.0;” casts the integer value to a double and divides the result by 100. This converts the data to the appropriate scale for the q-axis current.

Question 5

The vesc_packet.h file provides an interface for accessing the VESC driver’s functionalities. The purpose of declaring the current_qaxis() method in the vesc_packet.h file is to signal to other ROS nodes and subscribers that the driver can serve as a gate for publishing q-axis current data, and declares the method defined in the .cpp file.

The vesc_packet.h file establishes a standardized interface for accessing the functionality of the VESC driver, ensuring consistency and accessibility throughout the program. By maintaining

synchronization between the header file and the corresponding source file, it promotes clarity in code comprehension and simplifies maintenance tasks.

The modifications made in the .cpp file align with the objectives set in vesc_packet.h by providing a declaration for the method implemented within. This declaration ensures proper documentation and accessibility of the newly added functionality to other program components. Consequently, other parts of the program can utilize this functionality, enhancing overall system capability.

The declaration in the .cpp file facilitates the VESC driver's ability to publish this data by signaling to other ROS nodes or subscribers that it supports the publication of q-axis current data. This enables interested parties to subscribe to this data and receive updates whenever it changes. Moreover, it serves as documentation for developers, allowing them to leverage the functionality for publishing q-axis current data in their own ROS-based applications or systems.

Question 6

The command "values->current_qaxis()" is utilized to detect and retrieve data regarding the current in the q-axis, which is then transmitted and stored in the status of the q-axis current. The purpose of adding “state_msg->state.current_q = values->current_qaxis ()” to the “vesc_driver.cpp” file is to assign the q-axis current value from the VESC to the current_q state within the message. The VESC node ensures that the message published by the ROS system includes the q-axis current data. This allows all relevant information to be easily accessed by subscriber nodes from the VESC.

Question 7

By adding “current_q” to the “VescStateStamped” message, the information that the VESC node is expanded to include q-axis current data in addition to speed and current input. This aids subscribers and users to access this data for monitoring the motor’s performance and torque characteristics. This improvement offers a more detailed overview of the VESC's status, empowering subscribers to better observe and assess system performance. Subscribers can leverage the q-axis current data, in conjunction with other parameters, for activities such as monitoring motor performance, examining torque attributes, and executing control algorithms.

Question 8

The purpose of modifying the “arg name=“port”” line in the “vesc_driver_node.launch” file was to specify the port for communication between the VESC and the ROS system. By modifying the default value of this argument to "/dev/sensors/vesc", we are directing ROS to employ the designated port ("/dev/sensors/vesc") for communication with the VESC. This alteration guarantees that the VESC driver node establishes communication with the correct device file

associated with the VESC, which is crucial for establishing a reliable connection and facilitating data exchange between the ROS system and the VESC hardware. Moreover, this adjustment is in line with any udev rules previously configured to ensure the accurate identification of the VESC by the Jetson Nano, further streamlining communication and integration of the VESC into the ROS ecosystem on the McMaster AEV.

Objective 3: Experiment with VESC Using ROS

Question 9

$ rostopic pub -l /commands/motor/speed std_msgs/Float64 -- “7500.0”

“rostopic”: the rostopic library.

“pub”: command published data to a topic.

“-l”: topic is published in latch mode.

Latching mode

> `rostopic` will publish a message to `/topic_name` and keep it *latched* -- any new subscribers that come online after you start `rostopic` will hear this message. You can stop this at any time by pressing `ctrl-C`.

“/commands/motor/speed/std_msgs”: specifying the topic name. This designates the ROS topic where the message will be sent for publication. Specifically, the message will be directed to the /commands/motor/speed topic, which serves the purpose of regulating the motor's speed.

“/Float64”: specifying the topic type, float64. This segment specifies the type of message being transmitted. Within ROS, messages are categorized by their types, and here, the message type is denoted as std_msgs/Float64, indicating a floating-point number.

“7500.0”: data to be published. This denotes the specific message being broadcasted, indicating a floating-point value (Float64) that signifies the intended speed for the motor. In the given scenario, the speed directive is configured at 7500.0 RPM.

Objective 4: Controlling VESC with the Joystick

Question 10

Editing the “CMakeLists.txt” file and adding the “catkin_install_python” command was an important step. This ensures that Python is installed on the system, which we’ll need for the control program and initializing the system for the joystick. This CMakeLists.txt file specifies how the ROS package is built and what files are included in the package. Including the line catkin_install_python tells the system to install the Python development environment for future use. By incorporating the “catkin_install_python” command, we can indicate the inclusion of specific Python scripts during the package installation process. This ensures that upon building

and installing the package, these scripts are correctly distributed and made accessible for utilization.

Question 11

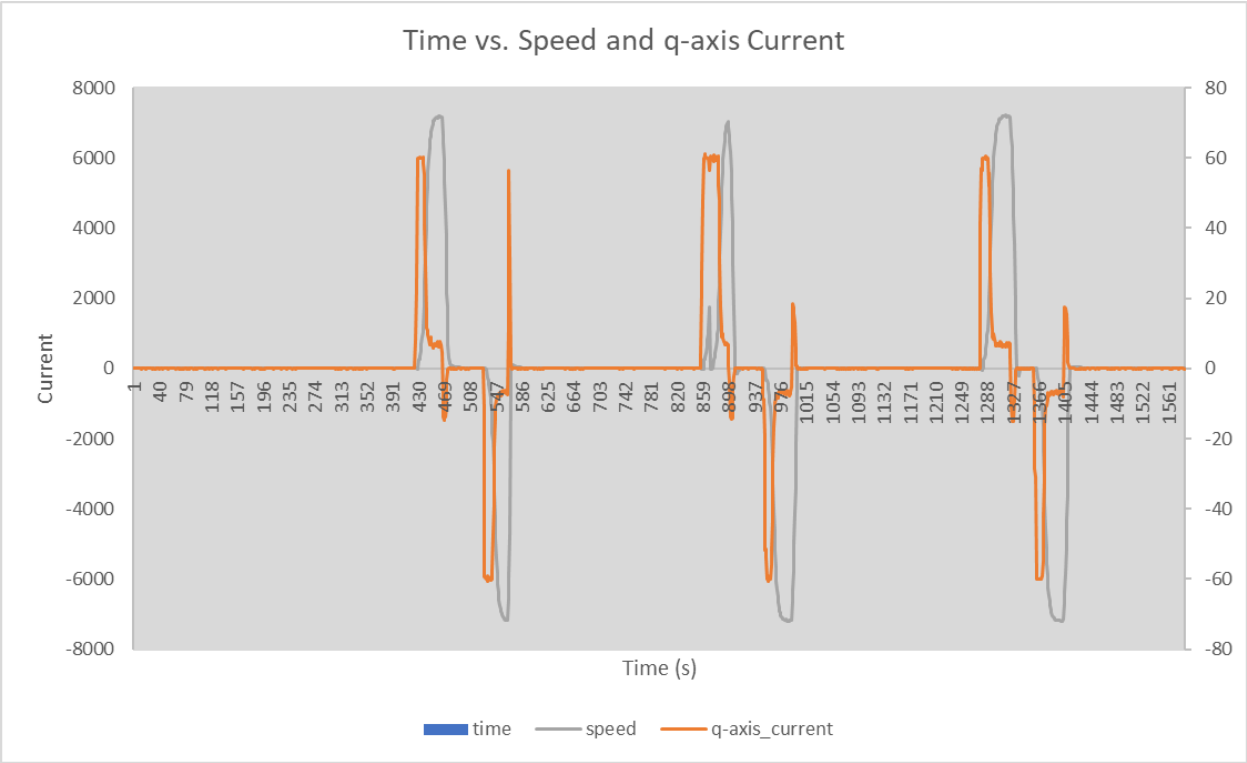
The controlled.launch file is a ROS launch configuration file. This launch file can create nodes and perform operations within the ROS environment. The <launch> tag indicates the beginning of a launch file. The <include file> tag includes additional launch files to be used and referred to in the launch file, including one to include joystick drivers and one for VESC drivers. The <node> tag launches a node given its name, package, and type. In our case, there are two <node> tags in this file. The first is of the name “JoyControl”, which can be found in the package called “test”, and of the type “JoyControl.py” which is a python script file. The second is named “servoControl” which is also in the package “test” and of the type “servoControl.py”. The <param> tag sets a parameter for a given node, that can be used to modify the node in a certain way. In this case there are two param tags called “Multiplier” and “ControlMethod”, which take the values of 0.12 and 0, respectively. The control method value may affect the AEV by setting it to it’s default setting [0] parameters.

Question 12

This line processes the input data given to the joystick from user interaction and converts it into an appropriate value for motor control. The way that it does this is by modifying the data given by data.axes[0], which returns the position of the joystick along the 0^{th} index which typically corresponds to the x-axis. We then multiply this value by -1 to invert the direction of the input data, then by 0.5 to scale it down to half of its original value range. Now instead of ranging from [-1,1] it ranges from [-0.5,0.5]. This helps to simplify the data given to the motor to control it, as well as use less resources in communicating the desired action from the joystick input. Finally, the range is shifted up by 0.5 such that there are no negative values, and the motor receives inputs ranging from [0,1] to operate which is commonly the input required for most motor controlling operations.

Objective 5: Logging Data from VESC

Question 13



The above graph shows the relationship between q-axis current and time. From lecture content, we know that the q-axis represents the torque output of a 3-phase induction motor. This is because this current is solely responsible for creating a rotating magnetic field that rotates the rotor and produces torque. In the data above, we interacted with the joystick in the following sequence: *up, stop, down, stop*. We see a direct relationship between the *up* direction and *positive current*, and the *down* direction and *negative current*. Specifically, when the vehicle moves forward, there is a positive RPM value observed, and when it moves backwards there is a negative RPM value observed. When the vehicle is not being operated through the joystick inputs, the current is steady state at 0, corresponding to 0 torque as torque is proportional to q-axis current. It is interesting to note that when the graph hits a *stop* in the sequence stated above, there is a sharp impulse current at that location. This demonstrates behaviour of regenerative breaking, where the motor acts simultaneously as a generator which can store amounts of kinetic energy after breaking as electrical energy to be used again when the vehicle is in motion. We observed that the vehicle was quick to stop as a result, as noted by the sharp slope in motion, both in the positive and the negative direction, which is a common characteristic of electric vehicles with regenerative breaking technology. We can very clearly observe that as speed increases, the q-axis current rises, which seems to reflect the motor's demand for additional torque while accelerating.